

Max Product of Three

First Algorithm: Triple Nested Loop (Non-Recursive)

Pseudo Code

```
FUNCTION WithForLoops(A[]):  
    n = length(A)  
    maxProduct = -INFINITY  
    FOR i FROM 0 TO n-3:  
        FOR j FROM i+1 TO n-2:  
            FOR k FROM j+1 TO n-1:  
                product = A[i] * A[j] * A[k]  
                IF product > maxProduct:  
                    maxProduct = product  
    RETURN maxProduct
```

Analysis & Complexity

Metric	Value
Time Complexity	$O(n^3)$
Space Complexity	$O(1)$

Second Algorithm: Sorting (Non-Recursive)

With Dual-Pivot QuickSort

Pseudo Code

```
FUNCTION WithSorting(A[]):  
    SORT(A)  
    n = length(A)  
    option1 = A[n-1] * A[n-2] * A[n-3]  
    option2 = A[0] * A[1] * A[n-1]  
    RETURN MAX(option1, option2)
```

Analysis & Complexity

Metric	Value
Time Complexity	$O(n \log n)$
Space Complexity	$O(1)$ or $O(\log n)$ for sort stack

Third Algorithm: Linear Tracking (Non-Recursive)

Pseudo Code

```
FUNCTION WithTracking(A[]):
    max1, max2, max3 = -INF, -INF, -INF
    min1, min2 = +INF, +INF
    FOR each num IN A:
        Update top-3 maximums (max1, max2, max3)
        Update bottom-2 minimums (min1, min2)
    option1 = max1 * max2 * max3
    option2 = max1 * min1 * min2
    RETURN MAX(option1, option2)
```

Analysis & Complexity

Metric	Value
Time Complexity	$O(n)$
Space Complexity	$O(1)$

Fourth Algorithm: Recursive Enumeration (Recursive)

Pseudo Code

```
FUNCTION maxProduct(nums[]):
    ans = -INFINITY
    CALL solve(nums, start=0, count=0, product=1)
    RETURN ans

FUNCTION solve(nums[], start, count, product):
    IF count == 3:
        ans = MAX(ans, product)
        RETURN
    FOR i FROM start TO length(nums)-1:
        CALL solve(nums, i+1, count+1, product * nums[i])

    ans = Long.MIN_VALUE;
    solve(nums, 0, 0, 1);
    return ans;
}
```

Analysis & Complexity

Metric	Value
Time Complexity	$O(n^3)$
Space Complexity	$O(n)$ due to recursion call stack

Comparison Between Algorithms

Algorithm	Type	Time Complexity	Space Complexity	Best For
Triple Loop (Brute Force)	Non-Recursive	$O(n^3)$	$O(1)$	Small arrays, simple understanding
Sorting	Non-Recursive	$O(n \log n)$	$O(1)$ or $O(\log n)$ for sort stack	Clean, readable code
Linear Tracking	Non-Recursive	$O(n)$	$O(1)$	Large arrays, best performance
Recursive Enumeration	Recursive	$O(n^3)$	$O(n)$ stack	Recursive practice / learning

- The Linear Tracking approach (Algorithm 3) is the clear winner for production use — $O(n)$ time and $O(1)$ space
- Non-recursive solutions generally outperform recursive ones here due to no stack overhead and simpler control flow